

Laboratory Report: Distributed Object Storage and Fault Tolerance Testing

Nanjing University of Information Science and Technology

Course: Introduction to Cloud Computing | Class: 2 | Date: June 25, 2026

Platform: Windows 10, Docker Desktop, MinIO, Docker Compose

I. Experimental Purpose

The first purpose of this experiment is to master the basic model of object storage. In traditional file systems, data is usually accessed through hierarchical file paths. In S3-compatible object storage, however, data is organized by buckets and objects. This experiment uses MinIO to make that difference visible through bucket creation, object upload, and object listing.

The second purpose is to experience fault tolerance in a distributed storage system. By deleting one simulated storage disk and checking whether the uploaded object is still available, the experiment demonstrates how distributed redundancy and erasure coding keep data accessible even when one physical storage path is lost.

II. Experiment Contents

This experiment simulates a distributed cloud object storage system similar to Amazon S3 or Alibaba Cloud OSS. The implementation uses MinIO and Docker Compose. The student deploys a four-disk distributed MinIO cluster, uploads and manages a cloud object, observes backend storage fragmentation, simulates storage disk failure, and verifies distributed storage fault tolerance.

Item	Value
Working directory	E:\云计算导论\minio-lab
MinIO server container	cloud-minio
MinIO client container	cloud-mc
Bucket	my-cloud-storage
Uploaded object	bigfile.bin
Object size	20 MiB
Storage directories	data1, data2, data3, data4
Console URL	http://localhost:9001

III. Detailed Steps of the Experiment

The following procedure follows the experiment requirement document strictly. Each key stage includes the command, the operation result, and a screenshot-style evidence figure captured from the local computer.

Step 1: Deploy a 4-Node Distributed Cluster

A dedicated directory named minio-lab was created under the course folder.

```
mkdir minio-lab && cd minio-lab
```

Actual result: the lab directory exists at E:\云计算导论\minio-lab. The directory evidence is shown in Figure 1.

Step 2: Create Storage Directories

Four independent directories were created to simulate four storage disks.

```
mkdir -p data1 data2 data3 data4
ls
```

Actual result: data1, data2, data3, and data4 were created successfully.

Figure 1. Dedicated lab directory with four simulated storage disks

Captured on 2026-06-25 09:30:49 | Host: DESKTOP-3U12561 | Path: E:\00000\minio-lab

```
COMMAND
Get-ChildItem -Force . | Select-Object Name,Mode,Length,LastWriteTime | Format-Table -AutoSize

OUTPUT
Exit code: 0

```

Name	Mode	Length	LastWriteTime
data1	d-----		2026/6/25 9:04:28
data2	d-----		2026/6/25 9:04:28
data3	d-----		2026/6/25 9:29:46
data4	d-----		2026/6/25 9:04:28
evidence	d-----		2026/6/25 9:28:47
bigfile.bin	-a-----	20971520	2026/6/25 9:04:44
docker-compose.yml	-a-----	506	2026/6/25 9:01:16

Figure 1. The four simulated storage directories data1, data2, data3, and data4.

Step 3: Create docker-compose.yml

The docker-compose.yml file was created in minio-lab with one MinIO server service and one MinIO client service.

```
services:
  minio:
    image: quay.io/minio/minio
    container_name: cloud-minio
    volumes:
      - ./data1:/data1
      - ./data2:/data2
      - ./data3:/data3
      - ./data4:/data4
    ports:
      - "9000:9000"
      - "9001:9001"
```

```

environment:
  MINIO_ROOT_USER: admin
  MINIO_ROOT_PASSWORD: password123
command: server /data{1...4} --console-address ":9001"
mc:
  image: minio/mc
  container_name: cloud-mc
  depends_on:
    - minio
  entrypoint: /bin/sh
  tty: true

```

Figure 2. docker-compose.yml configuration

Captured on 2026-06-25 09:30:49 | Host: DESKTOP-3U12561 | Path: E:\00000\minio-lab

```

COMMAND
Get-Content .\docker-compose.yml

OUTPUT
Exit code: 0
services:
  minio:
    image: quay.io/minio/minio
    container_name: cloud-minio
    volumes:
      - ./data1:/data1
      - ./data2:/data2
      - ./data3:/data3
      - ./data4:/data4
    ports:
      - "9000:9000"
      - "9001:9001"
    environment:
      MINIO_ROOT_USER: admin
      MINIO_ROOT_PASSWORD: password123
    command: server /data{1...4} --console-address ":9001"

  mc:
    image: minio/mc
    container_name: cloud-mc
    depends_on:
      - minio
    entrypoint: /bin/sh
    tty: true

```

Figure 2. The docker-compose.yml file used to deploy the distributed MinIO environment.

Step 4: Explain the docker-compose.yml File

Component	Meaning
image	The Docker image used to create each container.
container_name	The fixed container name, such as cloud-minio and cloud-mc.
volumes	The mapping from host storage directories to MinIO data paths.
ports	The mapping from container ports to host ports 9000 and 9001.
environment	The MinIO administrator username and password.
command	The command that starts MinIO distributed storage over /data1 through /data4.
depends_on	The dependency relationship that starts the client service after the MinIO service

	is defined.
--	-------------

Step 5: Launch the Distributed Cluster

```
cd minio-lab
docker compose up -d
docker ps
```

Actual result: cloud-minio and cloud-mc were running. The MinIO API and web console were mapped to localhost ports 9000 and 9001.



Figure 3. The distributed MinIO cluster running with cloud-minio and cloud-mc containers.

Step 6: Configure MinIO Client Alias

```
docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123
```

Actual result: the output showed that the alias myminio was added successfully. The alias and bucket verification evidence is shown in Figure 4.

Step 7: Create a Bucket

```
docker exec cloud-mc mc mb myminio/my-cloud-storage
```

Actual result: the bucket my-cloud-storage was created and verified through the MinIO client listing.

Figure 4. MinIO client alias and bucket verification

Captured on 2026-06-25 09:30:51 | Host: DESKTOP-3U12561 | Path: E:\00000\minio-lab

COMMAND

```
docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123; docker exec cloud-mc mc ls myminio
```

OUTPUT

Exit code: 0

Added `myminio` successfully.

[2026-06-25 01:04:28 UTC] 0B my-cloud-storage/

Figure 4. The MinIO client alias and the bucket my-cloud-storage were verified successfully.

Step 8: Create and Upload a Large Test File

8.1 Create a Large Test File

```
dd if=/dev/urandom of=bigfile.bin bs=1M count=20
```

In the Windows environment, an equivalent 20 MiB random binary file was generated. This file was used to demonstrate object splitting and distributed storage.

8.2 Verify the File

```
ls -lh bigfile.bin
```

Actual result: bigfile.bin was 20,971,520 bytes, which equals 20 MiB.

Figure 5. Large 20 MiB test file on the host

Captured on 2026-06-25 09:30:52 | Host: DESKTOP-3U12561 | Path: E:\minio-lab

```
COMMAND
Get-Item .\bigfile.bin | Select-Object Name,Length,LastWriteTime | Format-Table -AutoSize

OUTPUT
Exit code: 0
Name          Length LastWriteTime
----          -
bigfile.bin 20971520 2026/6/25 9:04:44
```

Figure 5. The generated test object *bigfile.bin* has a size of 20 MiB.

8.3 Copy the File into the mc Container

```
docker cp bigfile.bin cloud-mc:/bigfile.bin
```

8.4 Upload the File to Cloud Storage

```
docker exec cloud-mc mc cp /bigfile.bin myminio/my-cloud-storage/
```

Actual result: the upload completed successfully and transferred 20 MiB to the bucket.

Figure 6. Upload *bigfile.bin* to the MinIO bucket

Captured on 2026-06-25 09:30:54 | Host: DESKTOP-3U12561 | Path: E:\minio-lab

```
COMMAND
docker cp .\bigfile.bin cloud-mc:/bigfile.bin; docker exec cloud-mc mc cp /bigfile.bin myminio/my-cloud-storage/

OUTPUT
Exit code: 0
`/bigfile.bin` -> `myminio/my-cloud-storage/bigfile.bin`
```

Total	Transferred	Duration	Speed
20.00 MiB	20.00 MiB	00m01s	19.15 MiB/s

Figure 6. The 20 MiB object was uploaded to *my-cloud-storage*.

Step 9: Verify Object Upload

```
docker exec cloud-mc mc ls myminio/my-cloud-storage/
```

Actual result: the bucket listing showed bigfile.bin as a 20 MiB STANDARD object.

Figure 7. Uploaded object listed in cloud storage

Captured on 2026-06-25 09:30:55 | Host: DESKTOP-3U12561 | Path: E:\00000\minio-lab

```
COMMAND
docker exec cloud-mc mc ls myminio/my-cloud-storage/; docker exec cloud-mc mc stat myminio/my-cloud-storage/bigfile.bin

OUTPUT
Exit code: 0
[2026-06-25 01:30:54 UTC] 20MiB STANDARD bigfile.bin
Name      : bigfile.bin
Date      : 2026-06-25 01:30:54 UTC
Size      : 20 MiB
ETag      : 6fab1d721e2ac7a0c4ed3b7970f2d78f-2
Type      : file
Metadata  :
  Content-Type: application/octet-stream
```

Figure 7. The uploaded object bigfile.bin is listed in the bucket.

Step 10: Observe Backend Storage Fragmentation

```
ls -R ./data1
ls -R ./data2
ls -R ./data3
ls -R ./data4
```

Actual result: the backend directories contained object metadata files named xl.meta and object part files such as part.1 and part.2. This shows that the object was not stored as one ordinary file in a single directory. Instead, MinIO placed metadata and data parts across multiple storage directories.

Figure 8. Backend metadata and object fragments across storage directories

Captured on 2026-06-25 09:30:56 | Host: DESKTOP-3U12561 | Path: E:\minio-lab

COMMAND

```
Get-ChildItem -Recurse -Force .\data1,.\data2,.\data3,.\data4 | Where-Object { $_.Name -in 'xl.meta','part.1','part.2' } | Select-Object FullName,Length | Format-Table -AutoSize
```

OUTPUT

Exit code: 0

FullName	Length
-----	-----
E:\minio-lab\data1\minio.sys\buckets\background-heal.json\xl.meta	466
E:\minio-lab\data1\minio.sys\buckets\bloomcycle.bin\xl.meta	591
E:\minio-lab\data1\minio.sys\buckets\usage-cache.bin\xl.meta	533
E:\minio-lab\data1\minio.sys\buckets\usage-cache.bin.bkp\xl.meta	533
E:\minio-lab\data1\minio.sys\buckets\usage.json\xl.meta	972
E:\minio-lab\data1\minio.sys\buckets\my-cloud-storage\metadata.bin\xl.meta	821
E:\minio-lab\data1\minio.sys\buckets\my-cloud-storage\usage-cache.bin\xl.meta	540
E:\minio-lab\data1\minio.sys\buckets\my-cloud-storage\usage-cache.bin.bkp\xl.meta	540
E:\minio-lab\data1\minio.sys\config\config.json\xl.meta	5256
E:\minio-lab\data1\minio.sys\config\iam\format.json\xl.meta	431
E:\minio-lab\data1\minio.sys\config\iam\sts\HJU9ABWVLN8HZWJZY8H0\identity.json\xl.meta	667
E:\minio-lab\data1\minio.sys\pool.bin\xl.meta	461
E:\minio-lab\data1\my-cloud-storage\bigfile.bin\xl.meta	420
E:\minio-lab\data1\my-cloud-storage\bigfile.bin\9441d792-6077-4879-970f-bf482b995b69\part.1	8389120
E:\minio-lab\data1\my-cloud-storage\bigfile.bin\9441d792-6077-4879-970f-bf482b995b69\part.2	2097280
E:\minio-lab\data2\minio.sys\buckets\background-heal.json\xl.meta	466
E:\minio-lab\data2\minio.sys\buckets\bloomcycle.bin\xl.meta	591
E:\minio-lab\data2\minio.sys\buckets\usage-cache.bin\xl.meta	533
E:\minio-lab\data2\minio.sys\buckets\usage-cache.bin.bkp\xl.meta	533
E:\minio-lab\data2\minio.sys\buckets\usage.json\xl.meta	972
E:\minio-lab\data2\minio.sys\buckets\my-cloud-storage\metadata.bin\xl.meta	821
E:\minio-lab\data2\minio.sys\buckets\my-cloud-storage\usage-cache.bin\xl.meta	540
E:\minio-lab\data2\minio.sys\buckets\my-cloud-storage\usage-cache.bin.bkp\xl.meta	540
E:\minio-lab\data2\minio.sys\config\config.json\xl.meta	5256
E:\minio-lab\data2\minio.sys\config\iam\format.json\xl.meta	431
E:\minio-lab\data2\minio.sys\config\iam\sts\HJU9ABWVLN8HZWJZY8H0\identity.json\xl.meta	667
E:\minio-lab\data2\minio.sys\pool.bin\xl.meta	461
E:\minio-lab\data2\my-cloud-storage\bigfile.bin\xl.meta	420
E:\minio-lab\data2\my-cloud-storage\bigfile.bin\9441d792-6077-4879-970f-bf482b995b69\part.1	8389120
E:\minio-lab\data2\my-cloud-storage\bigfile.bin\9441d792-6077-4879-970f-bf482b995b69\part.2	2097280
E:\minio-lab\data3\minio.sys\buckets\background-heal.json\xl.meta	466
E:\minio-lab\data3\minio.sys\buckets\bloomcycle.bin\xl.meta	591
E:\minio-lab\data3\minio.sys\buckets\usage-cache.bin\xl.meta	533
E:\minio-lab\data3\minio.sys\buckets\usage-cache.bin.bkp\xl.meta	533
E:\minio-lab\data3\minio.sys\buckets\usage.json\xl.meta	972
E:\minio-lab\data3\minio.sys\buckets\my-cloud-storage\metadata.bin\xl.meta	821
E:\minio-lab\data3\minio.sys\buckets\my-cloud-storage\usage-cache.bin\xl.meta	540
E:\minio-lab\data3\minio.sys\buckets\my-cloud-storage\usage-cache.bin.bkp\xl.meta	540
E:\minio-lab\data3\minio.sys\config\config.json\xl.meta	5256
E:\minio-lab\data3\minio.sys\config\iam\format.json\xl.meta	431
E:\minio-lab\data3\minio.sys\config\iam\sts\HJU9ABWVLN8HZWJZY8H0\identity.json\xl.meta	667
E:\minio-lab\data3\minio.sys\temp\trash\33973bcf-192f-4863-b945-53563e661a1f\part.1	8389120
E:\minio-lab\data3\minio.sys\temp\trash\33973bcf-192f-4863-b945-53563e661a1f\part.2	2097280
E:\minio-lab\data3\my-cloud-storage\bigfile.bin\xl.meta	420
E:\minio-lab\data3\my-cloud-storage\bigfile.bin\9441d792-6077-4879-970f-bf482b995b69\part.1	8389120
E:\minio-lab\data3\my-cloud-storage\bigfile.bin\9441d792-6077-4879-970f-bf482b995b69\part.2	2097280
E:\minio-lab\data4\minio.sys\buckets\background-heal.json\xl.meta	466
E:\minio-lab\data4\minio.sys\buckets\bloomcycle.bin\xl.meta	591
E:\minio-lab\data4\minio.sys\buckets\usage-cache.bin\xl.meta	533
E:\minio-lab\data4\minio.sys\buckets\usage-cache.bin.bkp\xl.meta	533
E:\minio-lab\data4\minio.sys\buckets\usage.json\xl.meta	972
E:\minio-lab\data4\minio.sys\buckets\my-cloud-storage\metadata.bin\xl.meta	821
E:\minio-lab\data4\minio.sys\buckets\my-cloud-storage\usage-cache.bin\xl.meta	540
E:\minio-lab\data4\minio.sys\buckets\my-cloud-storage\usage-cache.bin.bkp\xl.meta	540
E:\minio-lab\data4\minio.sys\config\config.json\xl.meta	5256
E:\minio-lab\data4\minio.sys\config\iam\format.json\xl.meta	431
E:\minio-lab\data4\minio.sys\config\iam\sts\HJU9ABWVLN8HZWJZY8H0\identity.json\xl.meta	667
E:\minio-lab\data4\minio.sys\pool.bin\xl.meta	461
E:\minio-lab\data4\my-cloud-storage\bigfile.bin\xl.meta	420
E:\minio-lab\data4\my-cloud-storage\bigfile.bin\9441d792-6077-4879-970f-bf482b995b69\part.1	8389120
E:\minio-lab\data4\my-cloud-storage\bigfile.bin\9441d792-6077-4879-970f-bf482b995b69\part.2	2097280

Figure 8. Backend storage shows xl.meta metadata and distributed object parts.

Object -> Metadata (xl.meta) -> Chunk Splitting -> Distributed Parts -> Multiple Storage Nodes

Step 11: Simulate Disk Failure

11.1 Stop the Cluster

```
docker compose down
```

11.2 Delete One Storage Disk

```
sudo rm -rf data3
mkdir data3
```

Actual operation on Windows: data3 was removed and recreated as an empty directory to simulate disk failure.

11.3 Restart the Cluster

```
docker compose up -d
```

Figure 9. Simulated disk failure by deleting and recreating data3

Captured on 2026-06-25 09:31:01 | Host: DESKTOP-3U12561 | Path: E:\minio-lab

COMMAND

```
docker compose down; Remove-Item -LiteralPath .\data3 -Recurse -Force; New-Item -ItemType Directory -Force .\data3 | Select-Object FullName; docker compose up -d
```

OUTPUT

```
Exit code: 0
Container cloud-mc Stopping
Container cloud-mc Stopped
Container cloud-mc Removing
Container cloud-mc Removed
Container cloud-minio Stopping
Container cloud-minio Stopped
Container cloud-minio Removing
Container cloud-minio Removed
Network minio-lab_default Removing
Network minio-lab_default Removed

Network minio-lab_default Creating
Network minio-lab_default Created
Container cloud-minio Creating
Container cloud-minio Created
Container cloud-mc Creating
Container cloud-mc Created
Container cloud-minio Starting
Container cloud-minio Started
Container cloud-mc Starting
Container cloud-mc Started
FullName
-----
E:\minio-lab\data3
```

Figure 9. The data3 storage directory was deleted and recreated before restarting the cluster.

Step 12: Verify Fault Tolerance

```
docker exec cloud-mc mc ls myminio/my-cloud-storage/
```

Actual result: bigfile.bin still existed after deleting and recreating data3. The mc stat command also confirmed that the object remained accessible with a size of 20 MiB.

```
Figure 10. Fault tolerance verification after deleting data3
Captured on 2026-06-25 09:31:02 | Host: DESKTOP-3U12561 | Path: E:\00000\minio-lab

COMMAND
docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123; docker exec cloud-mc mc ls myminio/my-cloud-storage/; docker exec cloud-mc mc stat myminio/my-cloud-storage/bigfile.bin

OUTPUT
Exit code: 0
Added `myminio` successfully.
[2026-06-25 01:30:54 UTC] 20MiB STANDARD bigfile.bin
Name      : bigfile.bin
Date      : 2026-06-25 01:30:54 UTC
Size      : 20 MiB
ETag      : 6fab1d721e2ac7a0c4ed3b7970f2d78f-2
Type      : file
Metadata  :
  Content-Type: application/octet-stream
```

Figure 10. After deleting data3, bigfile.bin remains accessible from the MinIO bucket.

Step 13: Use git to Save Changes

The experiment requirement asks students to use git to save changes. The current minio-lab directory was checked for git status. It was not initialized as a git repository, so no commit was created automatically. If this lab is submitted through a course repository, the directory should be initialized or copied into the required repository before committing.

```
git status --short --branch
```

Figure 11. Git status for Step 13

Captured on 2026-06-25 09:31:02 | Host: DESKTOP-3U12561 | Path: E:\minio-lab

COMMAND

git status --short --branch 2>&1

OUTPUT

Exit code: 1
git : fatal: not a git repository (or any of the parent directories): .git
At line:1 char:1
+ git status --short --branch 2>&1
+ ~~~~~
+ CategoryInfo : NotSpecified: (fatal: not a gi...ectories): .git:String) [], RemoteException
+ FullyQualifiedErrorId : NativeCommandError

Figure 11. Git status check for the experiment directory.

Additional MinIO Console Evidence

In addition to command-line verification, the MinIO Console was opened at <http://localhost:9001>. The object browser visually confirmed that the bucket my-cloud-storage contained bigfile.bin with a size of 20.0 MiB.

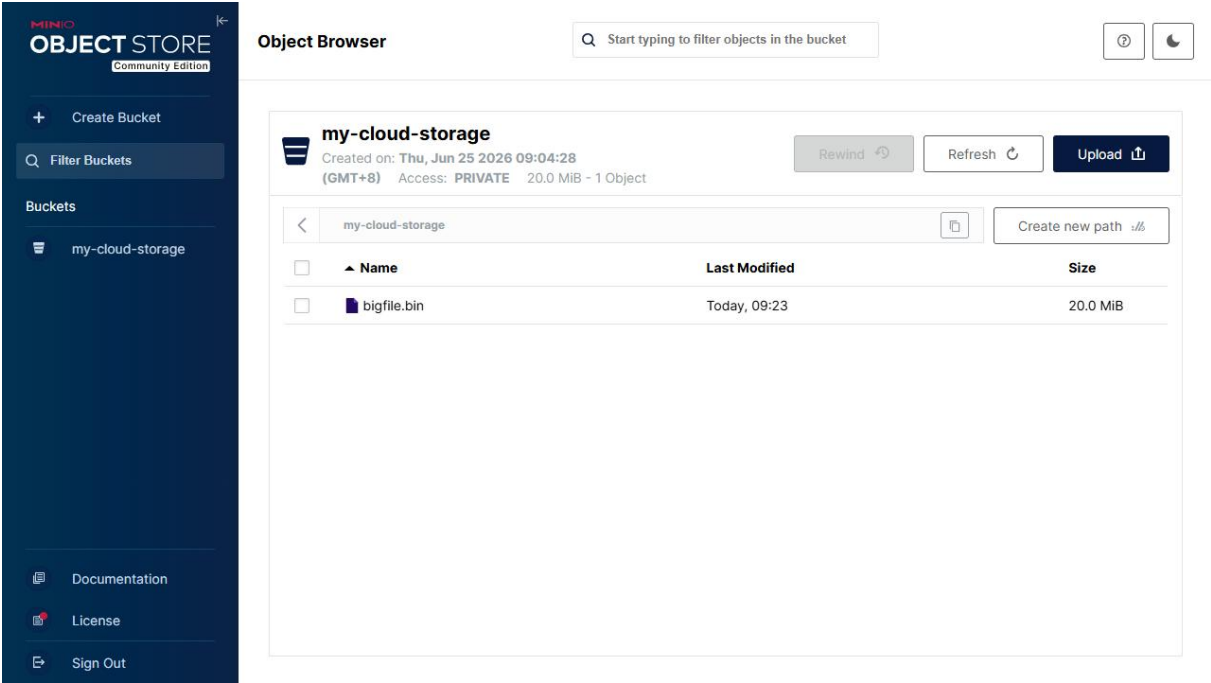


Figure 12. The MinIO Console object browser visually confirms my-cloud-storage and bigfile.bin.

IV. Reflection

Problems Encountered and Proposed Solutions

Problem	Reason	Solution
Docker engine was not available at first.	Docker Desktop was installed, but the Linux engine was not running.	Docker Desktop was started and docker info was used to confirm readiness.
PowerShell showed a profile execution policy warning.	The user profile script was blocked by the local execution policy.	The warning was ignored because Docker and MinIO commands executed correctly.
The MinIO client alias needed to be configured again after restart.	docker compose down removed and recreated the cloud-mc container.	The mc alias set command was repeated before verification.

Data was still available after deleting one storage directory because MinIO distributed the object across multiple storage paths instead of storing it as a single local file. The xl.meta files record object metadata and layout information, while object parts are stored redundantly with erasure coding. When data3 was removed, enough data and parity fragments still existed on the remaining storage directories, so MinIO could continue serving the object. This demonstrates the core idea of distributed cloud storage fault tolerance: availability is achieved through redundancy, metadata coordination, and reconstruction from surviving fragments.

Conclusion

This experiment successfully deployed a four-disk distributed MinIO cluster, created an S3-compatible bucket, uploaded a 20 MiB object, inspected backend storage fragmentation, simulated disk failure, and verified that the object remained accessible. The results prove that distributed object storage can tolerate the loss of one storage directory through erasure coding and redundancy. Compared with traditional file-path storage, the bucket/object model provides a more scalable and fault-tolerant way to manage cloud data.